

PROPOSAL PACKAGE | DOCUMENT 03

Solution Architecture

High-level plan and low-level design for the AI Executive Intelligence Assistant, the CRM + Follow-up System, and the shared platform that connects them

Document	03 of 15
Audience	Client CTO / IT, procurement, and the delivery team
Version	1.0 (first-meeting edition)
Date	September 2026
Classification	Confidential - prepared for the client

Contents

1. Purpose and how to read this document.....	3
2. The requirement, restated precisely.....	3
2.1 System 1: AI Executive Intelligence Assistant.....	3
2.2 System 2: CRM + Follow-up System.....	3
2.3 Cross-cutting asks.....	4
3. High-level architecture.....	4
3.1 Why a shared platform layer.....	5
3.2 Shared platform components.....	5
3.3 System 1 at a glance.....	6
3.4 System 2 at a glance.....	7
3.5 Deployment and data residency.....	7
4. How the two systems connect at the top level.....	8
5. Plugging in real data later without rebuilding.....	8
6. High-level delivery plan.....	9
PART B: LOW-LEVEL DESIGN.....	11
7. Component design.....	11
7.1 Runtime components.....	11
7.2 Service boundaries and independence.....	11
8. Data model (low level).....	12
8.1 Platform schema.....	12
8.2 Intelligence schema.....	12
8.3 CRM schema.....	13
9. Processing pipelines.....	13
9.1 Ingestion pipeline (System 1).....	13
9.2 Answering pipeline (System 1).....	14
9.3 Outreach approval workflow (System 2).....	15
9.4 Reminders.....	15
10. Interfaces.....	15
10.1 REST API (selected endpoints).....	15
10.2 Connectors.....	16
11. Environments, deployment and operations.....	16
12. Quality, evaluation and safety controls.....	17
12.1 Evaluation approach.....	17
12.2 Safety controls specific to the assistant.....	17
12.3 Safety controls specific to the CRM.....	17
13. Non-functional requirements summary.....	18

1. Purpose and how to read this document

This document has two halves. **Part A (sections 2-6)** is the high-level plan: what we build, why it is shaped this way, and how the two systems stay independent yet connected. It is written for executives and non-technical readers. **Part B (sections 7-13)** is the low-level design: components, data model, processing pipelines, interfaces, environments, and quality controls. It is written for the client's technical reviewers and for our delivery team, and it is the blueprint we will build from.

Design principles that govern every decision in this document:

1. **Closed domain.** The assistant answers only from documents the client has loaded. No open-internet answers.
2. **Evidence first.** Every answer cites document and page. No citation, no answer.
3. **Human in control.** No outbound message is sent without a named human approving it.
4. **Configuration over code.** Stages, categories, checklists, fields and connectors are configured, so real data plugs in later without a rebuild.
5. **Portability.** The client owns the code, the cloud accounts and the data; AI models are swappable behind a gateway.

2. The requirement, restated precisely

2.1 System 1: AI Executive Intelligence Assistant

#	Requirement (client wording)	What it means in engineering terms
R1	Retrieve specific info from documents on request, controlled access, not open internet	Retrieval-augmented generation (RAG) over a private corpus, with per-user, per-document access control enforced before retrieval, and the model forbidden from using outside knowledge.
R2	Flag contradictions where two documents state different figures, dates or terms for the same thing	A structured claims ledger: extract (entity, attribute, value, unit, date, source, page, quote) from every document, normalise, then compare like-for-like across documents; surface conflicts with both sources shown.
R3	Track missing information: gaps where a decision needs a document or data point not yet provided	Configurable decision checklists (required evidence per decision type) matched against the ledger and document inventory; plus detection of references to documents that are not in the corpus (for example "see Annex B").
R4	Answer management questions in plain language based only on the loaded document set	Grounded answer generation with citations, a second-pass grounding check that verifies each sentence is supported, and explicit refusal when evidence is absent.

2.2 System 2: CRM + Follow-up System

#	Requirement (client wording)	What it means in engineering terms
C1	Three categories: investors; partners/counterparties;	Relationship records typed by category, each with its own configurable pipeline and stage set; a GCC pipeline template reflecting introduction,

#	Requirement (client wording)	What it means in engineering terms
	customers/members; GCC-specific relationship pipeline	trust-building and regulatory steps.
C2	Contact and relationship records with stage/status tracking	Organisation and Person entities, Relationship records with stage, owner, status, last touch, next action; full history of stage changes.
C3	Follow-up scheduling and reminders	Follow-up tasks with due dates and owners; reminders to staff by email/Teams/in-app; overdue escalation.
C4	Any outreach must go through human approval before sending; no fully automated outbound	Outreach Request workflow: draft, review queue, approve/reject with reason, send via connected mailbox under the approver's authority, immutable log. No scheduler may send.
C5	Basic reporting: who has been contacted, stage, overdue	Dashboards and exportable reports: contacts touched by period, pipeline by stage and category, overdue follow-ups, dormant relationships, approval turnaround.

2.3 Cross-cutting asks

- **Build approach and tools for each** (section 3 and Document 06).
- **Structure that accepts real data later without rebuild** (section 5 and Document 07).
- **Timeline to a working structure/demo** (section 6 and Document 04).
- **Cost or pricing model** (Document 05 workbook and Document 14 draft statement of work).

3. High-level architecture

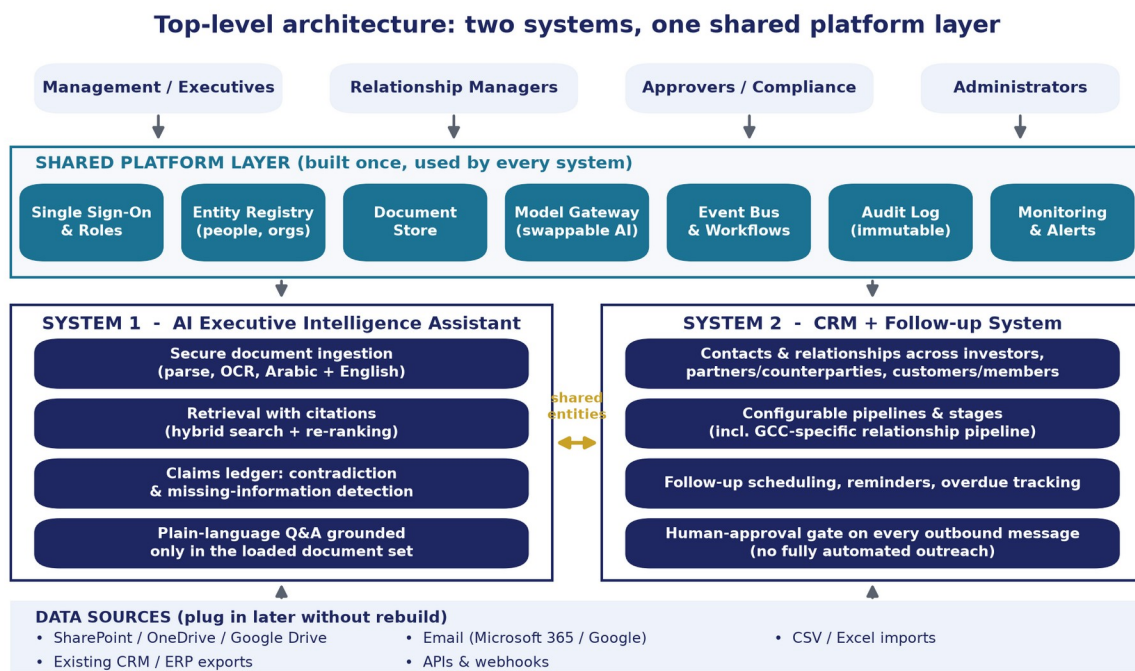


Figure 1. Top-level architecture: two systems on one shared platform layer

3.1 Why a shared platform layer

Both systems need the same seven things: who the user is and what they may see; a single list of organisations and people; somewhere to keep documents; a way to call AI models; a way to run workflows and reminders; a tamper-evident audit trail; and monitoring. Building these once avoids two identity systems, two contact lists and two audit logs. It also means a third or fourth system (for example a board-pack generator or a compliance tracker) can be added later on the same base.

Each product remains **individually deployable and individually usable**. The CRM works with the Executive Intelligence Assistant switched off, and vice versa. The connection is at the top level only: shared identity, shared entities, shared audit, and the ability for the assistant to read CRM attachments as documents and for the CRM to show relevant assistant findings on a relationship record.

3.2 Shared platform components

Component	Responsibility	Technology (pilot -> production)
Identity and access	Single sign-on, roles (executive, relationship manager, approver, analyst, admin), per-document and per-record permissions	Supabase Auth with OIDC -> Microsoft Entra ID / Okta; Keycloak if self-hosted
Entity registry	Master list of organisations and people with de-duplication and merge; used by both systems	PostgreSQL tables with matching rules
Document store	Original files, versions, checksums, classification tags, access-control lists	Supabase Storage -> S3 / Azure Blob in a GCC region, or on-premises object storage
Model gateway	One internal API for all AI calls; routes by policy (document classification, cost, residency); logs tokens and cost; swaps providers without code change	LiteLLM proxy (open source) in front of Anthropic API / AWS Bedrock / Microsoft Foundry / in-region vLLM
Workflow and events	Background jobs (ingestion, extraction, reminders), event bus so systems react to each other	PostgreSQL job queue -> Redis + workers; Temporal if workflows become complex
Audit log	Append-only record of every read, answer, approval and send, with hash chaining	PostgreSQL append-only table with periodic hash anchoring; export to SIEM at enterprise tier
Monitoring	Application metrics, AI tracing (prompts, retrieved passages, latencies, cost), alerting	Langfuse (open source LLM tracing), OpenTelemetry, Grafana / Sentry

3.3 System 1 at a glance

System 1: how a document becomes a trusted, cited answer

INGESTION (happens once per document)



STORAGE: Postgres + pgvector (chunks, embeddings, claims ledger, access-control tags, versions) | object storage for originals

ANSWERING (happens on every question)



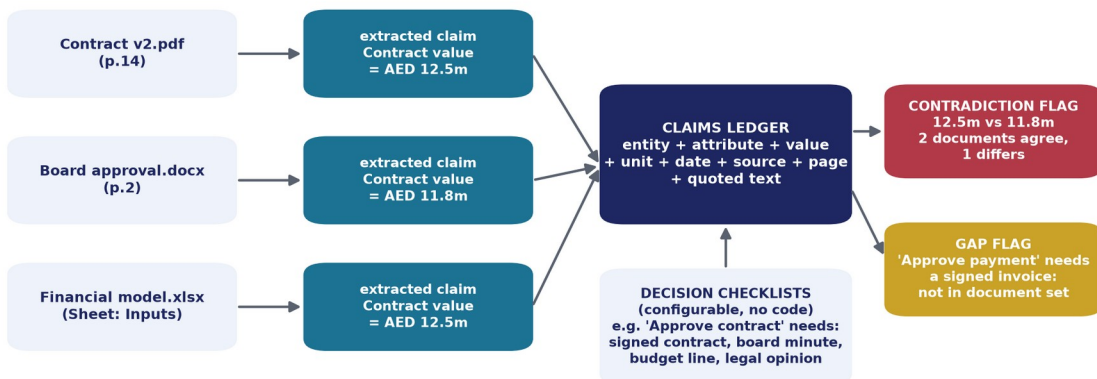
Design rules: the model may only answer from retrieved passages; every answer carries document + page citations; unsupported answers are refused, not guessed; every query, passage, answer and user is logged to an immutable audit trail.

Figure 2. How a document becomes a trusted, cited answer

Two flows. **Ingestion** runs once per document version: scan, classify, parse (including OCR for scans, in Arabic and English), chunk with page references, embed and index, and extract claims into the ledger.

Answering runs on every question: apply the user's permissions, retrieve by keyword and meaning, re-rank, draft an answer with citations, check grounding, and either deliver or decline. Everything is logged.

Contradiction and gap detection: the claims ledger



Why this matters: plain chatbots cannot reliably spot conflicts because they see only a few passages at a time. A structured ledger compares every figure, date and term across all documents, records versions (v1 vs v2 is not a contradiction), and routes each flag to a named owner for resolution.

Figure 3. Contradiction and gap detection through the claims ledger

The claims ledger is the part that makes this more than a chatbot. Extraction turns prose into rows: "Contract value = AED 12.5m (Contract v2.pdf, p.14)". Comparison then finds the row from the board minute that says AED 11.8m. Decision checklists list what evidence each decision type needs and mark what is present, what is missing, and what is contradicted.

3.4 System 2 at a glance

System 2: human-approval gate on all outbound communication

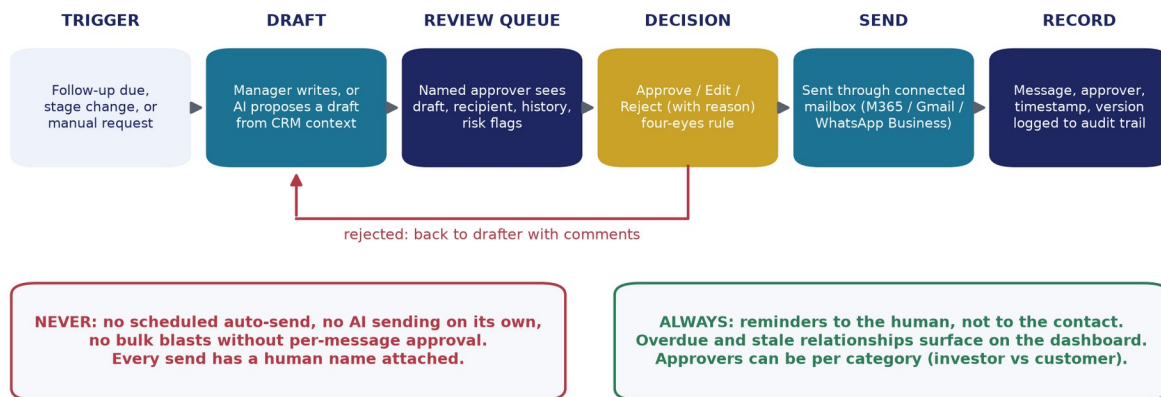
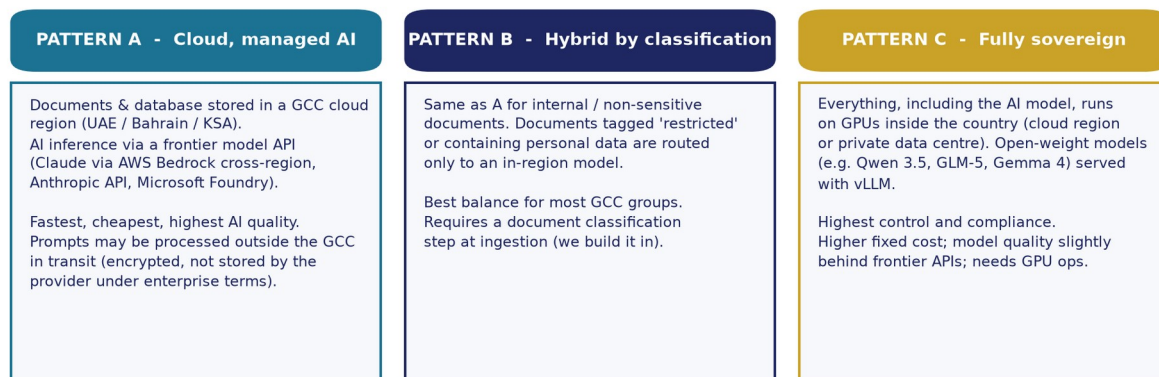


Figure 4. Human-approval gate on all outbound communication

The CRM is deliberately conventional in its records and pipelines, and deliberately strict in its outbound behaviour. Staff draft (optionally with AI assistance from the relationship's history), an approver reviews, and only then is the message sent through the organisation's own mailbox, so replies land where they always did. Reminders go to staff, never to contacts.

3.5 Deployment and data residency

Where the data and the AI model live: three deployment patterns



The model gateway makes the pattern a configuration choice, not a rebuild: the same application code talks to any model. Recommendation: start on Pattern A for the demo, design for B, keep C as a documented upgrade path.

Figure 5. Three deployment patterns for data and model residency

The client will need to decide where data at rest and AI processing may take place. The architecture supports three patterns, selectable by configuration:

- **Pattern A, cloud with managed AI.** Data at rest in a GCC cloud region (for example AWS UAE me-central-1 or Bahrain me-south-1, Azure UAE North, Google Cloud Dammam). AI calls go to a frontier model provider. As of early 2026, Claude models are reachable from the AWS Middle East regions through Amazon Bedrock's global cross-region inference, which means prompts may be processed outside the region in transit under AWS encryption and enterprise terms. This is the fastest and cheapest pattern and gives the highest model quality.
- **Pattern B, hybrid by classification.** Same as A for internal and non-sensitive material; documents tagged restricted, or containing personal data, are routed by the model gateway only to an in-region model. Our recommended target state for most GCC groups.
- **Pattern C, fully sovereign.** All processing, including the model, runs on GPUs inside the country, either in a hyperscaler's local region or a private data centre, using open-weight models (for example Qwen 3.5, GLM-5, Gemma 4) served with vLLM. Highest control; higher fixed cost; model quality a step behind frontier APIs.

We recommend **starting on Pattern A for the demo, designing for Pattern B, and keeping Pattern C as a documented upgrade path.** The gateway makes this a configuration change, not a rebuild.

4. How the two systems connect at the top level

Connection	Behaviour
Shared identity	One login; roles apply across both systems; an approver in the CRM can be a read-only user in the assistant.
Shared entity registry	The "Al Futtaim Investment LLC" in a contract is the same organisation record as the investor in the pipeline. The assistant links claims to organisations; the CRM shows those claims on the relationship page.
Documents flow one way	Attachments added to a CRM interaction (a signed NDA, a term sheet) can be pushed into the assistant's corpus with the same access tags as the relationship.
Findings flow the other way	Contradictions or gaps that involve a counterparty appear as alerts on that relationship. Example: "The MoU date in the CRM (12 March) differs from the signed MoU (14 March)."
Shared audit	One tamper-evident trail. A regulator or internal auditor sees who asked what, who approved what, and what was sent, in one place.
Event bus	Stage changes, new documents, approvals and reminders are events; either system can subscribe. New automations do not require touching the other system's code.

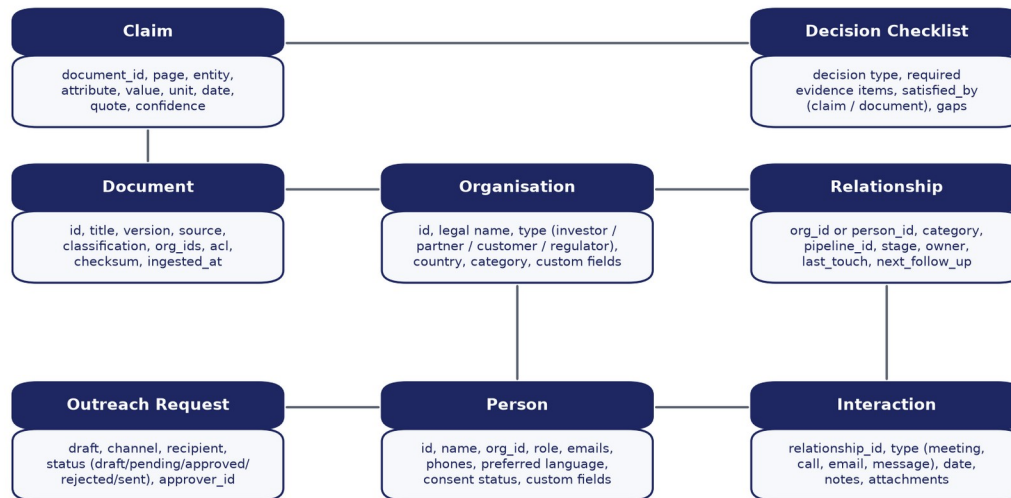
5. Plugging in real data later without rebuilding

The client asked for a structure that accepts real data later. This is achieved by five design choices, detailed in Document 07:

1. **Schema-first data model** with a JSON "custom fields" column on every core table, so new attributes are configuration, not migrations.
2. **Configuration-driven pipelines, stages, categories and decision checklists**, editable by an administrator in the interface.

3. **Import templates** (CSV/XLSX with column mapping and validation preview) for contacts, organisations, relationships and historical interactions.
4. **Connectors** for SharePoint/OneDrive, Google Drive, Microsoft 365 and Gmail mailboxes, and an API with webhooks for anything else.
5. **Demo data is flagged**, not baked in. Every demo record carries an "is_demo" marker; one administrative action removes all of it and leaves the configuration intact.

Shared data model (simplified): why real data plugs in without a rebuild



Every table carries: tenant_id, created_by, created_at, updated_at, is_demo (so demo data can be wiped in one command), and a JSON 'custom fields' column so client-specific attributes are added by configuration. Imports map client columns to these fields through a template, not code.

Figure 6. Shared data model (simplified)

6. High-level delivery plan

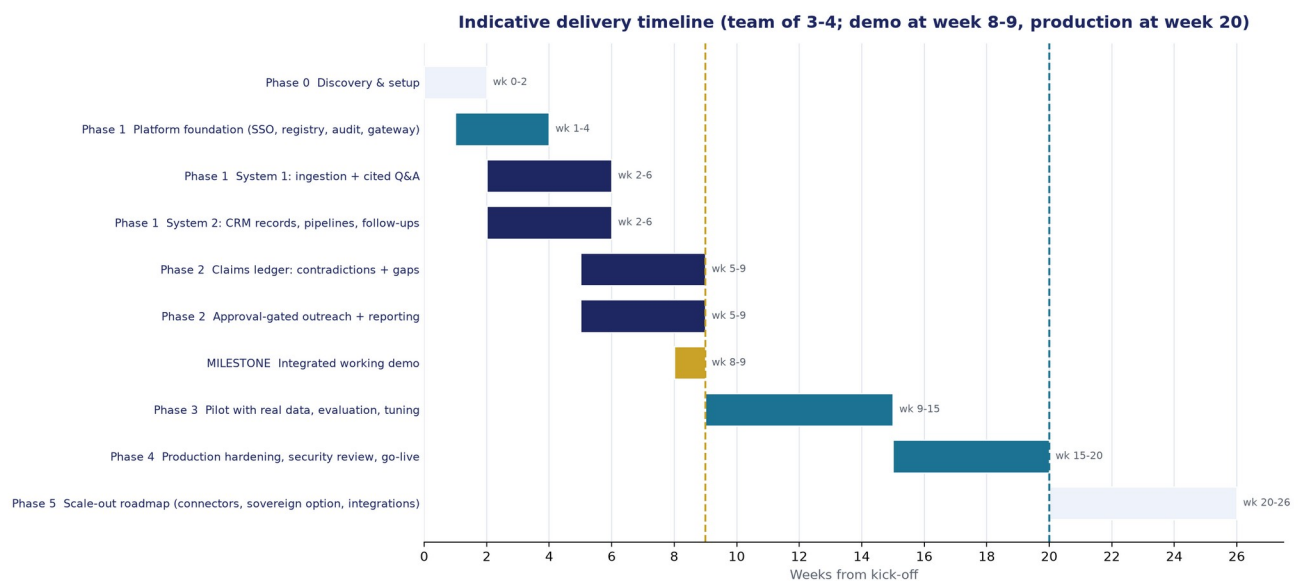


Figure 7. Indicative timeline

Phase	Weeks	Outcome
0 Discovery	0-2	Confirmed scope, jurisdictions, residency pattern, identity provider, sample documents, decision types, pipeline stages. Environments created in the client's cloud account.
1 Foundation and working structure	1-6	Platform layer live; System 1 ingests documents and answers with citations; System 2 has records, pipelines and follow-ups. Sample data loaded.
2 Verification layer and approval gate	5-9	Claims ledger with contradiction and gap detection; decision checklists; outreach approval queue; reports. Integrated working demo at week 8-9.
3 Pilot with real data	9-15	Real documents and contacts; evaluation against the agreed golden question set; tuning of extraction and retrieval; user training.
4 Production hardening	15-20	Security review, penetration test, backups and disaster recovery, runbooks, SLAs, go-live.
5 Scale-out	20+	Additional connectors, sovereign model option, ERP/DMS/board-portal integration, multi-entity tenancy.

The full plan with staffing, dependencies and acceptance criteria is in Document 04.

PART B: LOW-LEVEL DESIGN

7. Component design

7.1 Runtime components

Component	Language / runtime	Responsibilities and notes
Web application	Next.js (React, TypeScript)	Single sign-on, assistant chat and findings pages, CRM pages (contacts, pipeline board, follow-ups, approval queue, dashboards), administration (configuration, imports, connectors). Responsive; works in mobile browsers. Arabic right-to-left layout supported.
Core API	Node.js (TypeScript)	Business logic for CRM and platform: entities, relationships, tasks, outreach workflow, audit, configuration. Exposes REST endpoints and webhooks.
Intelligence service	Python (FastAPI)	Document ingestion, parsing and OCR orchestration, chunking, embedding, claims extraction, retrieval, answer generation, grounding check, contradiction and gap detection. Python is used because the document and AI tooling ecosystem is strongest there.
Workers	Python and Node	Background jobs: ingestion pipeline, scheduled extraction re-runs, reminder generation, connector syncs. Pull from a job queue; horizontally scalable.
Model gateway	LiteLLM (Python)	Single OpenAI-compatible endpoint for chat, embeddings and re-ranking; routes to configured providers; enforces per-classification routing, budgets, rate limits; logs usage.
Database	PostgreSQL 16 with pgvector	System of record for both systems; vectors and full-text indexes; row-level security.
Object storage	S3-compatible	Original documents and derived artefacts (page images, parsed text), versioned, encrypted.
Cache / queue	Redis	Job queue, rate-limit counters, short-lived session data. (Pilot uses a PostgreSQL-backed queue to avoid an extra service.)
Observability	Langfuse, OpenTelemetry, Grafana or Sentry	AI traces (prompt, retrieved chunks, answer, cost), application logs, metrics, alerts.

7.2 Service boundaries and independence

- The **CRM** depends only on the Core API, the database and the platform layer. It does not call the intelligence service except for optional draft suggestions, which are behind a feature flag.
- The **assistant** depends on the intelligence service, the model gateway, the database and the platform layer. It reads entity records from the Core API read model; it never writes CRM data.

- Each service has its own database schema (namespace) inside one PostgreSQL instance during pilot and can be split into separate databases at enterprise scale without code changes, because cross-schema access goes through APIs rather than joins.

8. Data model (low level)

All tables carry: id (UUID), tenant_id, created_at, created_by, updated_at, updated_by, is_demo, custom (JSONB). Soft-delete via deleted_at where retention rules require.

8.1 Platform schema

Table	Key columns
users	email, display_name, idp_subject, status, locale (en/ar), last_login_at
roles, user_roles	role code (executive, relationship_manager, approver, analyst, admin), scope (category or business unit)
organisations	legal_name, short_name, type (investor, partner, counterparty, customer, member, regulator, vendor), country, jurisdiction, registration_no, parent_id, aliases (text[]), classification
persons	full_name, full_name_ar, organisation_id, title, emails (jsonb), phones (jsonb), preferred_language, preferred_channel, consent_status, consent_source, consent_at
documents	title, source (upload, sharepoint, drive, email, crm), source_ref, version, supersedes_id, mime, size, checksum, classification (public, internal, confidential, restricted), acl (jsonb), organisation_ids (uuid[]), language(s), status (received, scanned, parsed, indexed, extracted, failed), ingested_at
audit_events	actor_id, action, object_type, object_id, details (jsonb), ip, user_agent, occurred_at, prev_hash, hash (append-only; hash chain anchored daily)
configuration	key, value (jsonb), version, changed_by, changed_at (pipelines, stages, categories, checklists, routing policies)
jobs	type, payload, status, attempts, run_after, locked_by, error

8.2 Intelligence schema

Table	Key columns
document_pages	document_id, page_no, text, text_ar, ocr_confidence, image_ref
chunks	document_id, page_start, page_end, section_path, text, token_count, embedding (vector), tsv (full-text), acl (copied from document for pre-filtering)
claims	document_id, page_no, entity_text, entity_org_id, attribute (normalised code, e.g. contract_value, effective_date, term_months, interest_rate, counterparty), value_raw, value_num, value_text, value_date, unit, currency, as_of_date, quote, confidence, extractor_model, extracted_at
claim_links	claim_id_a, claim_id_b, relation (agrees, conflicts, supersedes, unclear), delta, detected_at
findings	type (contradiction, gap, dangling_reference, stale), severity, title, description, claim_ids,

Table	Key columns
	document_ids, decision_id, owner_id, status (open, acknowledged, resolved, dismissed), resolution_note
decisions	type (approve_contract, release_payment, board_resolution, investor_commitment, custom), name, organisation_id, target_date, owner_id
checklist_templates, checklist_items	decision type; required evidence item (e.g. signed contract, board minute, budget line, legal opinion, KYC file); matching rule (document type tag, attribute presence, or manual)
queries	user_id, question, language, retrieved_chunk_ids, rerank_scores, answer, citations (jsonb), grounding_result, model, tokens_in, tokens_out, cost, latency_ms, feedback

8.3 CRM schema

Table	Key columns
pipelines	name, category (investor, partner, customer), stage list (ordered, with codes, SLA days, exit criteria), is_default
relationships	organisation_id or person_id, category, pipeline_id, stage_code, status (active, dormant, closed_won, closed_lost, on_hold), owner_id, secondary_owner_id, priority, source, introduced_by_person_id, last_touch_at, next_follow_up_at, health_score
stage_history	relationship_id, from_stage, to_stage, changed_by, changed_at, note
interactions	relationship_id, type (meeting, call, email_in, email_out, message, event, note), occurred_at, participants (uuid[]), summary, sentiment, attachments (document ids)
follow_ups	relationship_id, title, due_at, owner_id, status (open, done, snoozed, cancelled), reminder_policy, completed_at
outreach_requests	relationship_id, recipient_person_id, channel (email, whatsapp, sms, letter), subject, body, body_lang, drafted_by, draft_source (human, ai_assisted), status (draft, pending_approval, changes_requested, approved, rejected, sent, failed), approver_id, decided_at, decision_note, sent_at, provider_message_id, version
outreach_versions	outreach_request_id, version, body, edited_by, edited_at
approval_policies	category, min_approvers, approver_role, escalation_after_hours, restricted_recipient_rules

9. Processing pipelines

9.1 Ingestion pipeline (System 1)

1. **Receive.** Upload via UI, or connector sync (SharePoint/OneDrive/Drive/mailbox). Compute checksum; if identical to an existing document, link rather than duplicate. Detect version relationships by filename pattern and metadata; administrator can confirm "supersedes".
2. **Scan and classify.** Anti-malware scan (ClamAV or cloud-native). Classification by rule (source folder, filename) and by a lightweight model pass; administrator can override. Classification drives the model-routing policy.

3. **Parse.** Digital PDFs, Word, Excel, PowerPoint and email via Docling (open source, runs locally, strong table extraction). Scanned pages via OCR with Arabic and English support (Azure AI Document Intelligence in the Azure UAE region, or Tesseract/PaddleOCR self-hosted for the sovereign pattern). Excel: each sheet is parsed into cell-level text with sheet and cell references so citations can point to "Sheet Inputs, cell C14".
4. **Chunk.** Structure-aware chunking (headings, clauses, table rows) with 300-600 tokens per chunk and overlap; each chunk records page range and section path. Contract clauses are kept whole where possible.
5. **Embed and index.** Multilingual embeddings (Voyage voyage-3.5 via API, or BGE-M3 self-hosted); stored in pgvector with an HNSW index. Full-text index built in parallel for exact terms, numbers and names.
6. **Extract claims.** A structured-output prompt to the extraction model (Claude Sonnet 5 for cost, Claude Opus 5 for complex financial documents) returns claims as JSON validated against a schema. Values are normalised (currency to ISO code, numbers to base units, dates to ISO 8601, Arabic numerals handled). Each claim keeps the verbatim quote and page.
7. **Compare.** For each new claim, find claims with the same normalised entity and attribute across other documents. Numeric: conflict if the difference exceeds a tolerance (configurable, default 0.5%). Dates: conflict if different, unless one document supersedes the other. Text terms: an LLM adjudication pass decides agrees / conflicts / unclear with a rationale. Conflicts become findings.
8. **Check gaps.** Re-evaluate decision checklists that reference the affected organisations or decision types; update missing-evidence findings. Detect dangling references ("Annex B", "the schedule attached") with no matching document.
9. **Notify and audit.** Owners of affected findings are notified; every step logs to audit with timings and model usage.

9.2 Answering pipeline (System 1)

1. **Authorise.** Resolve the user's permitted document set (roles plus per-document ACLs). This filter is applied inside the retrieval query, never after.
2. **Understand the question.** Detect language (Arabic or English); expand with synonyms and normalised entity names from the registry; identify if the question is about a figure, a date, a term, a status or a comparison.
3. **Retrieve.** Hybrid retrieval: top 40 by vector similarity and top 40 by full-text match, fused (reciprocal rank fusion). If the question names an entity or attribute that exists in the claims ledger, ledger rows are added as high-priority evidence.
4. **Re-rank.** A cross-encoder re-ranker (Voyage rerank-2.5 or self-hosted bge-reranker-v2-m3) orders the fused set; the top 8-12 passages go to the model.
5. **Generate.** The answer model (Claude Opus 5 by default; Claude Sonnet 5 for routine lookups; in-region model under Pattern B/C when classification requires) receives the passages with document titles and pages, and a system instruction that forbids outside knowledge and requires a citation for each statement. Structured output returns the answer, a list of citations, and a confidence label.
6. **Ground.** A second, independent pass checks each sentence of the answer against the cited passages and marks supported / partially supported / unsupported. Unsupported sentences are removed; if nothing remains, the assistant replies that the loaded documents do not contain the answer and lists the closest documents found.
7. **Attach findings.** If any cited claim is part of an open contradiction, the answer shows the conflict inline ("Note: another document states a different value").

8. **Deliver and log.** Answer, citations, retrieved passages, model, tokens, cost and latency are logged. Users can rate the answer; ratings feed the evaluation set.

Target latency: first token under 3 seconds, full answer under 15 seconds for typical questions.

9.3 Outreach approval workflow (System 2)

1. **Trigger:** follow-up due, stage change, or manual "compose".
2. **Draft:** the relationship manager writes the message, or requests an AI-assisted draft. The draft generator sees only the relationship history, the contact's language preference and approved templates; it cannot send.
3. **Submit:** the draft enters the approval queue with recipient, channel, history summary, and automated risk flags (new recipient domain, restricted counterparty, missing consent, attachment classification).
4. **Decide:** an approver (per approval policy: role, category, minimum approvers) approves, requests changes, or rejects with a reason. Four-eyes rule: drafter cannot approve their own message. Escalation after a configurable number of hours.
5. **Send:** on approval, the message is sent through the connected mailbox or channel (Microsoft Graph send-as, Gmail API, WhatsApp Business Platform) using the approver's authority. The system has no scheduled send capability by design; approval and sending are the same human action.
6. **Record:** full message, versions, approver, timestamps, provider message id and delivery status are written to the interaction history and the audit log. Replies received in the connected mailbox are attached to the relationship.

9.4 Reminders

A scheduled job evaluates follow-ups daily and hourly: due today, overdue, and stale relationships (no touch in N days per stage). Reminders go to staff via in-app notification, email digest, and optionally Microsoft Teams or Slack. Reminders never go to external contacts.

10. Interfaces

10.1 REST API (selected endpoints)

Endpoint	Purpose
POST /v1/documents ; GET /v1/documents/{id} ; POST /v1/documents/{id}/reindex	Upload, inspect, reprocess documents
POST /v1/ask	Ask a question; returns answer, citations, findings, trace id
GET /v1/findings ; PATCH /v1/findings/{id}	List and resolve contradictions and gaps
GET /v1/decisions/{id}/checklist	Evidence status for a decision
POST /v1/organisations ; POST /v1/persons ; POST /v1/relationships	Create records; bulk endpoints for imports
PATCH /v1/relationships/{id}/stage	Move stage with note; writes history

Endpoint	Purpose
POST /v1/follow-ups ; PATCH /v1/follow-ups/{id}	Schedule and complete follow-ups
POST /v1/outreach ; POST /v1/outreach/{id}/submit ; POST /v1/outreach/{id}/decide	Draft, submit for approval, approve/reject
GET /v1/reports/pipeline ; /v1/reports/overdue ; /v1/reports/activity	Reporting data for dashboards and exports
POST /v1/imports ; GET /v1/imports/{id}/preview	Import with mapping and validation preview
Webhooks: document.indexed, finding.created, relationship.stage_changed, outreach.sent	Event notifications for integrations

All endpoints require a bearer token from the identity provider; service-to-service calls use short-lived signed tokens. Rate limits and request logging apply.

10.2 Connectors

Connector	Notes
SharePoint / OneDrive	Microsoft Graph, delta sync per library; permissions mirrored to document ACLs
Google Drive	Drive API changes feed; shared drives supported
Microsoft 365 mailbox	Graph: read selected folders for ingestion; send-as for approved outreach
Gmail / Google Workspace	Gmail API equivalent
WhatsApp Business Platform	Meta Cloud API; template messages; approval gate applies
CSV / XLSX import	Contacts, organisations, relationships, interactions, documents manifest
Generic API and webhooks	For ERP, existing CRM or document management system exports

11. Environments, deployment and operations

Aspect	Pilot (Tier 0-1)	Enterprise / sovereign (Tier 2-3)
Hosting	Vercel (web) + Supabase (PostgreSQL, storage, auth) or a single cloud account in a GCC region; containers for the intelligence service	Kubernetes or managed containers (AWS ECS/EKS, Azure AKS) in a GCC region or private data centre; GPU node pool for in-region models
Environments	dev, demo, pilot	dev, test, staging, production, plus disaster-recovery region
Infrastructure as code	Terraform from day one; the client's account,	Terraform; policy-as-code; change approval

Aspect	Pilot (Tier 0-1)	Enterprise / sovereign (Tier 2-3)
	our automation	
CI/CD	GitHub Actions: tests, security scans, container build, deploy	Same, plus signed images, manual production gate
Backups	Daily automated, 30-day retention	Point-in-time recovery, cross-region copies, quarterly restore tests
Availability target	Business hours, best effort	99.9% monthly, defined RTO/RPO (for example 4 hours / 15 minutes)
Support	Email, next business day	Tiered SLA, on-call for severity 1

12. Quality, evaluation and safety controls

12.1 Evaluation approach

Large language models are probabilistic. The system is engineered so that errors are visible and measurable rather than hidden:

- **Golden question set.** During discovery we agree 100-200 real questions with known answers and source pages. Every release is scored on retrieval hit rate, citation accuracy, answer faithfulness and refusal correctness (refusing when the corpus genuinely lacks the answer).
- **Claims extraction accuracy.** A labelled sample of documents gives precision and recall for extracted figures, dates and terms; target precision above 95% for numeric fields at pilot exit.
- **Contradiction detection.** Seeded conflicts in the pilot corpus measure detection rate and false-positive rate; every flag shows both sources, so a false positive costs a minute, not a decision.
- **Human feedback loop.** Thumbs up/down and "wrong source" reports on every answer feed the evaluation set; findings resolved as "not a contradiction" tune tolerance rules.
- **Regression gate.** No release ships if the golden-set score drops.

12.2 Safety controls specific to the assistant

- System instruction forbids outside knowledge and requires citations; grounding check enforces it.
- Prompt-injection defence: document text is treated as data, never as instructions; retrieved passages are delimited and the model is told to ignore instructions inside them; high-risk phrases are flagged in ingestion.
- Per-user permission filter applied before retrieval; the model never sees passages the user may not read.
- No training on client data. Providers are used under enterprise terms with zero or limited retention where available; the self-hosted pattern removes third-party processing entirely.
- Rate limits, spend budgets and anomaly alerts on the model gateway.

12.3 Safety controls specific to the CRM

- No automated path to external contacts exists in the code base; the send function requires an approval record signed by a different user than the drafter. Reminders and digests go to staff only.

- Restricted-recipient rules (sanctioned or embargoed counterparties, regulators) block submission until a compliance role approves.
- Consent status is displayed on every contact; outreach to contacts without recorded consent raises a flag.
- Full message history is immutable; edits create versions.

13. Non-functional requirements summary

Requirement	Target
Languages	English and Arabic in interface, documents and answers; right-to-left layout
Document formats	PDF (digital and scanned), DOCX, XLSX, PPTX, EML/MSG, images (JPG/PNG scans)
Corpus size	Pilot: up to 5,000 documents / 150,000 pages; enterprise: millions of pages with the same architecture
Concurrency	Pilot: 25 concurrent users; enterprise: 500+ with horizontal scaling
Security	Encryption in transit (TLS 1.2+) and at rest (AES-256); SSO with MFA; RBAC and row-level security; audit hash chain; secrets in a managed vault; annual penetration test
Privacy	Data minimisation; consent tracking; retention and deletion policies; data subject request support; residency pattern per jurisdiction (Document 08)
Accessibility	WCAG 2.1 AA for the web application
Portability	Client owns repositories and cloud accounts; containerised services; no proprietary database features that block migration